RBE 500 — Foundations of Robotics

Project Assignment 3

Worcester Polytechnic Institute

Tracy Yang Kyle Yee

July 27, 2026

1 Velocity-Level Kinematics (Part 1)

1.1 Overview

For part 1, a new node was implemented that serves two functions: taking joint velocities and converting them to end effector velocities, and taking end effector velocities and converting them to joint velocities.

1.2 Service Definitions

Two new services were created in the `velocity_kinematics` package.

Listing 1: JointToEEVelocity.srv

```
float64 theta1_dot
float64 theta2_dot
float64 d3_dot
---
float64 vx
float64 vy
float64 vz
bool success
string message
```

Listing 2: EEToJointVelocity.srv

```
float64 vx
float64 vy
float64 vz
---
float64 theta1_dot
float64 theta2_dot
float64 d3_dot
bool success
```

```
string message
```

1.3 Velocity Kinematics Node

Listing 3: velocity_kinematics_node.py

```python
import rclpy
import numpy as np
from rclpy.node import Node
from sensor_msgs.msg import JointState
from velocity_kinematics.srv import JointToEEVelocity,
   EEToJointVelocity

A1 = 0.40
A2 = 0.30


def jacobian(theta1, theta2):
    s1, c1 = np.sin(theta1), np.cos(theta1)
    s12, c12 = np.sin(theta1 + theta2), np.cos(theta1 + theta2)
    return np.array([
        [-A1*s1 - A2*s12, -A2*s12, 0],
        [ A1*c1 + A2*c12,  A2*c12, 0],
        [ 0,               0,      1]
    ])


class VelocityKinematicsNode(Node):

    def __init__(self):
        super().__init__('velocity_kinematics_node')
        self.theta1 = 0.0
        self.theta2 = 0.0

        self.create_subscription(JointState, '/joint_states', self.
           joint_state_cb, 10)
        self.create_service(JointToEEVelocity, '/scara/
           joint_to_ee_velocity', self.fwd_vel_cb)
        self.create_service(EEToJointVelocity, '/scara/
           ee_to_joint_velocity', self.inv_vel_cb)

    def joint_state_cb(self, msg):
        if 'joint_1' in msg.name:
            self.theta1 = msg.position[msg.name.index('joint_1')]
        if 'joint_2' in msg.name:
            self.theta2 = msg.position[msg.name.index('joint_2')]

```

```
    def fwd_vel_cb(self, request, response):
        qdot = [request.theta1_dot, request.theta2_dot, request.
           d3_dot]
        vx, vy, vz = jacobian(self.theta1, self.theta2) @ qdot
        response.vx, response.vy, response.vz = vx, vy, vz
        response.success = True
        return response

    def inv_vel_cb(self, request, response):
        xdot = [request.vx, request.vy, request.vz]
        Jv = jacobian(self.theta1, self.theta2)
        try:
            q1, q2, q3 = np.linalg.solve(Jv, xdot)
        except np.linalg.LinAlgError:
            response.theta1_dot, response.theta2_dot, response.
               d3_dot = 0.0, 0.0, 0.0
            response.success = False
            response.message = (
                f'Singular Jacobian at theta1={self.theta1:.4f},
                   theta2={self.theta2:.4f} '
                '(theta2 near 0 or pi) -- no unique joint-velocity
                   solution.'
            )
            self.get_logger().warn(response.message)
            return response
        response.theta1_dot, response.theta2_dot, response.d3_dot =
           q1, q2, q3
        response.success = True
        return response


def main(args=None):
    rclpy.init(args=args)
    node = VelocityKinematicsNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

In the velocity kinematics node, there are two main functions: forward velocity callback and inverse velocity callback. These functions take theta 1 and theta 2 from a joint state subscriber and calculate the jacobian using a separate function. With the jacobian, the forward velocity function multiplies it with qdot to get vx, vy, and vz. The inverse velocity function solves for qdot in the same equation. This equation can fail if the determinant of Jv is 0

(at $\theta_2 = 0$ or π, the arm's fully-extended or fully-folded configuration), so `np.linalg.solve` is wrapped in a `try/except` on `LinAlgError`: on a singular Jacobian the service returns `success = false` with a diagnostic message instead of letting the exception propagate and crash the node. This matters beyond Part 1 itself – Part 4's constant-velocity node (Section 4) calls this service every cycle and depends on it failing gracefully rather than taking the whole service down.

2 Extending the Position Controller (Part 2)

2.1 Overview

In part 2, the URDF and controller node were updated to extend the position controller from part 2.

2.2 URDF Changes

The joint types for joints 1 and 2 were changed back from fixed to revolute. The Gazebo actuation plugin block was then updated so all three joints are driven by a real effort input.

Listing 4: joint_1 and joint_2 reverted to revolute

```
<joint name="joint_1" type="revolute">
    <parent link="base_link"/><child link="link_1"/>
    <origin xyz="0 0 0.5"/><axis xyz="0 0 1"/>
    <limit effort="1000" lower="-1.57" upper="1.57" velocity="0.5"/>
</joint>

<joint name="joint_2" type="revolute">
    <parent link="arm_1"/><child link="link_2"/>
    <origin xyz="0.4 0 0"/><axis xyz="0 0 1"/>
    <limit effort="1000" lower="-1.57" upper="1.57" velocity="0.5"/>
    <dynamics damping="0.2" friction="0.1"/>
</joint>
```

Actuation plugin debugging. The first attempt used Gazebo's `JointPositionController` plugin per joint, with the controller node publishing its computed PD effort onto that plugin's command topic. In simulation the joints simply refused to move: the commanded values (tens of N·m) were being interpreted as absolute *position* targets rather than effort, and since those targets fell far outside each joint's ± 1.57 rad limit the plugin applied no torque at all. Setting `use_force_commands` on that plugin did not change this behavior. The fix was to drop `JointPositionController` entirely and drive every joint through a single `JointController` plugin per joint in force mode – the same plugin type used for Part 3 – confirmed by publishing a raw effort value directly to its command topic and observing the joint move exactly as expected. Both the position controller (Part 2) and the velocity controller (Part 3) now publish onto the same `/scara/jointX_effort_cmd` topic per joint;

only one of the two ROS nodes is ever run at a time; running both simultaneously would have them fight over which effort value the single plugin instance actually applies.

Listing 5: Single force-mode actuation plugin per joint

```
<gazebo>
    <plugin filename="gz-sim-joint-state-publisher-system"
            name="gz::sim::systems::JointStatePublisher">
        <topic>/world/empty/model/scara/joint_state</topic>
        <joint_name>joint_1</joint_name>
        <joint_name>joint_2</joint_name>
        <joint_name>joint_3</joint_name>
    </plugin>

    <plugin filename="gz-sim-joint-controller-system"
            name="gz::sim::systems::JointController">
        <joint_name>joint_1</joint_name>
        <use_force_commands>true</use_force_commands>
        <topic>/scara/joint1_effort_cmd</topic>
        <p_gain>1</p_gain><i_gain>0</i_gain>
    </plugin>
    <plugin filename="gz-sim-joint-controller-system"
            name="gz::sim::systems::JointController">
        <joint_name>joint_2</joint_name>
        <use_force_commands>true</use_force_commands>
        <topic>/scara/joint2_effort_cmd</topic>
        <p_gain>1</p_gain><i_gain>0</i_gain>
    </plugin>
    <plugin filename="gz-sim-joint-controller-system"
            name="gz::sim::systems::JointController">
        <joint_name>joint_3</joint_name>
        <use_force_commands>true</use_force_commands>
        <topic>/scara/joint3_effort_cmd</topic>
        <p_gain>1</p_gain><i_gain>0</i_gain>
    </plugin>
</gazebo>
```

2.3 Controller Node Changes

To reuse the same position controller from part 2, it had to be adjusted to work for multiple joints. This was done by creating dictionaries for each of the stored values. Now, each of the variables that were originally just for a singular value have values for each of the joints.

Listing 6: Per-joint state, publishers, and services

```
JOINT_NAMES = ['joint_1', 'joint_2', 'joint_3']

KP = {'joint_1': 50.0, 'joint_2': 50.0, 'joint_3': 800.0}
KD = {'joint_1': 5.0,  'joint_2': 5.0,  'joint_3': 80.0}
```

```
JOINT_MIN = {'joint_1': -1.57, 'joint_2': -1.57, 'joint_3': -0.20}
JOINT_MAX = {'joint_1':  1.57, 'joint_2':  1.57, 'joint_3':  0.20}

self.ref_position = {j: 0.0 for j in JOINT_NAMES}
self.cur_position = {j: 0.0 for j in JOINT_NAMES}
self.prev_error   = {j: 0.0 for j in JOINT_NAMES}

self.cmd_pub = {
    j: self.create_publisher(Float64, f'/scara/{j.replace("_","")}_effort_cmd', 10)
    for j in JOINT_NAMES
}

self.srv = {
    j: self.create_service(
        SetJointPosition,
        f'/scara/set_{j.replace("_","")}_position',
        self.make_set_position_cb(j)
    )
    for j in JOINT_NAMES
}
```

Listing 7: Callback factory — binds a joint name to its own service callback

```
def make_set_position_cb(self, joint_name):
    def set_position_cb(request, response):
        target = request.position
        if not (JOINT_MIN[joint_name] <= target <= JOINT_MAX[joint_name]):
            response.success = False
            response.message = f'Requested {target:.4f} outside limits for {joint_name}.'
            return response
        self.ref_position[joint_name] = target
        response.success = True
        response.message = f'{joint_name} reference set to {target:.4f}'
        return response
    return set_position_cb
```

Listing 8: PD control loop, generalized to all three joints

```
def control_loop(self):
    now = self.get_clock().now()
    if self.prev_time is None:
        self.prev_time = now
        return
    dt = (now - self.prev_time).nanoseconds * 1e-9
    if dt <= 0.0:
```

```
            return
        self.prev_time = now

        for j in JOINT_NAMES:
            error = self.ref_position[j] - self.cur_position[j]
            error_dot = (error - self.prev_error[j]) / dt
            self.prev_error[j] = error

            effort = KP[j] * error + KD[j] * error_dot
            effort = max(-EFFORT_LIMIT, min(EFFORT_LIMIT, effort))

            msg = Float64()
            msg.data = effort
            self.cmd_pub[j].publish(msg)
```

3 Velocity Controllers (Part 3)

3.1 Overview

Each joint gets a PI velocity controller rather than reusing the Part 2 PD law:

$$\tau = K_p\, e_v + K_i \int e_v\, dt, \qquad e_v = \dot{q}_{ref} - \dot{q}_{cur}.$$

Differentiating the velocity signal (as PD would) amplifies measurement noise; the integral term instead removes steady-state error without a derivative term.

The controller subscribes to `/joint_states` for measured velocity and to per-joint `/scara/jointX_vel_ref` topics for the reference, publishing effort on the same `/scara/jointX_effort_cmd` topics used in Part 2 (only one of the two ROS controllers runs at a time).

3.2 Velocity Controller Node

Listing 9: PI gains and per-joint state/topics

```
JOINT_NAMES = ['joint_1', 'joint_2', 'joint_3']

KP = {'joint_1': 20.0, 'joint_2': 0.5, 'joint_3': 8.0}
KI = {'joint_1': 2.0,  'joint_2': 0.4, 'joint_3': 1.0}

self.ref_velocity = {j: 0.0 for j in JOINT_NAMES}
self.cur_velocity = {j: 0.0 for j in JOINT_NAMES}
self.integral     = {j: 0.0 for j in JOINT_NAMES}

self.ref_subs = {
    j: self.create_subscription(
```

```
        Float64, f'/scara/{j.replace("_", "")}_vel_ref', self.
            make_ref_cb(j), 10)
    for j in JOINT_NAMES
}
self.cmd_pub = {
    j: self.create_publisher(Float64, f'/scara/{j.replace("_", "")}
        _effort_cmd', 10)
    for j in JOINT_NAMES
}
```

Listing 10: PI control loop

```
for j in JOINT_NAMES:
    error = self.ref_velocity[j] - self.cur_velocity[j]

    self.integral[j] += error * dt
    self.integral[j] = max(-INTEGRAL_LIMIT, min(INTEGRAL_LIMIT, self
        .integral[j]))

    effort = KP[j] * error + KI[j] * self.integral[j]
    effort = max(-EFFORT_LIMIT, min(EFFORT_LIMIT, effort))

    msg = Float64()
    msg.data = effort
    self.cmd_pub[j].publish(msg)
```

The integral term is clamped (anti-windup) so a large or long-lived error – e.g. while a joint accelerates toward a step reference – cannot saturate the integrator and cause overshoot once the error shrinks.

3.3 Gain Tuning

Gains were first tuned per joint in isolation, temporarily setting the other two joints' URDF types to `fixed` (reverted afterward). Equal gains (K_p=20, K_i=2) worked for joint_1 and joint_2 alone; joint_3 needed a much larger pair (K_p=400, K_i=40).

Running all three joints together for Part 4 exposed a problem isolated tuning missed: joint_2 and joint_3 chattered against their $\pm$0.5 rad/s limits instead of settling, while joint_1 tracked smoothly. The cause is inertia mismatch – link_2's I_{zz}=0.01 is $\sim$50$\times$ smaller than link_1's (I_{zz}=0.5), and link_3 is only 0.1 kg – so the same effort gain that gives joint_1 a modest, damped acceleration gives joint_2/3 a huge one, overshooting the limit and oscillating against it. Scaling gains down roughly with each joint's effective inertia (to K_p=0.5, K_i=0.4 for joint_2) killed the chatter.

That fix went too far, though. With chatter gone, Figure 1 still showed joint_2's measured velocity visibly lagging its reference by a growing gap that never closed within the test window, while joint_1 and joint_3 tracked cleanly. The difference is what each joint is being

asked to track: joint_1 and joint_3's Part 4 references are nearly constant, so the step-response-style gains from isolated tuning still apply, but joint_2's reference is a continuously-decreasing ramp – holding a constant $+y$ Cartesian velocity requires θ_2 to keep shrinking (see Part 4) – and gains damped enough to avoid step-input chatter were too sluggish to keep up with a moving target. joint_2's gains were raised back up from that low point until the lag closed, without reintroducing the original chattering:

	joint_1	joint_2	joint_3
K_p	20.0	3.0	8.0
K_i	2.0	1.5	1.0

Lesson: gains tuned on a single isolated joint against a step input aren't guaranteed stable, or even sufficient, once the full coupled system runs together against a reference that keeps moving – both need to be re-checked against the actual Part 4 conditions, not just the isolated step-response test from Part 3.

4 Constant Velocity Reference (Part 4)

4.1 Overview

Part 4 drives the end effector in a straight $+y$ line by holding a constant Cartesian velocity $(\dot{x}, \dot{y}, \dot{z}) = (0, v_y, 0)$ and converting it to joint rates via the Part 1 inverse velocity service. Since the Jacobian depends on the current configuration, this is re-solved on a timer as the robot moves rather than solved once.

Listing 11: constant_velocity_node.py — Part 4

```
JOINT_NAMES = ['joint_1', 'joint_2', 'joint_3']
VY = 0.05  # m/s, constant end-effector velocity in +y

class ConstantVelocityNode(Node):

    def __init__(self):
        super().__init__('constant_velocity_node')
        self.cli = self.create_client(EEToJointVelocity, '/scara/
           ee_to_joint_velocity')
        self.ref_pub = {
            j: self.create_publisher(Float64, f'/scara/{j.replace("_
               ","")}_vel_ref', 10)
            for j in JOINT_NAMES
        }
        self.pending = False
        self.timer = self.create_timer(0.05, self.timer_cb)  # 20 Hz

    def timer_cb(self):
        if self.pending:
            return
```

```
        req = EEToJointVelocity.Request()
        req.vx, req.vy, req.vz = 0.0, VY, 0.0
        self.pending = True
        future = self.cli.call_async(req)
        future.add_done_callback(self.response_cb)

    def response_cb(self, future):
        self.pending = False
        resp = future.result()
        if not resp.success:
            self.get_logger().warn('Inverse velocity solve failed (
               near singularity?)')
            return
        qdot = {'joint_1': resp.theta1_dot, 'joint_2': resp.
           theta2_dot, 'joint_3': resp.d3_dot}
        for j in JOINT_NAMES:
            self.ref_pub[j].publish(Float64(data=qdot[j]))
```

The service call is asynchronous (`call_async` with a `done_callback`) since a blocking wait inside the same executor would deadlock the node against its own response.

Two failure modes are handled. First, $\det(J_v) = a_1 a_2 \sin\theta_2 = 0$ at θ_2=0: Part 1 now returns `success = false` instead of raising a linear-algebra exception, and this node just skips that cycle's reference rather than crashing – so the robot must first be moved off θ_2=0 using the Part 2 position controller. Second, a constant $+y$ velocity can itself drive θ_2 back toward zero over time, so required joint rates grow the longer the reference is held and the arm can hit joint limits within seconds. The results below use a short window from a start pose well clear of θ_2=0; a robust implementation would detect this drift and stop or replan before saturating.

4.2 Logging and Plotting

The Part 3 velocity controller logs reference and measured velocity for all three joints to `velocity_log.csv` at each 100 Hz control cycle, in the same format as the Part 2 position log.

Listing 12: `plot_velocity.m`

```
data = readmatrix(fullfile(script_dir, 'velocity_log.csv'));
t = data(:,1);
for i = 1:3
    ref = data(:, 2*i);
    cur = data(:, 2*i + 1);
    subplot(3,1,i);
    plot(t, ref, 'r--', 'LineWidth', 1.5); hold on;
    plot(t, cur, 'b-', 'LineWidth', 1.5);
    grid on; xlabel('Time (s)'); legend('Reference', 'Measured');
end
```

4.3 Results

Figure 1: Reference vs. measured joint velocity for all three joints while tracking a constant $+y$ end-effector velocity of $v_y = 0.05$ m/s.